

## SC EXAM PREP — PREDICTED 2026

# 暗号・認証・PKI

PKCE／パスキー(FIDO2/WebAuthn)／OIDC／OCSP／mTLS／耐量子暗号(PQC)。認証・認可・鍵基盤の頻出論点を、代表5問で「経路」と「役割」から体系学習する。

**Q1・RFC 7636**

**PKCE** と認可コード横取り  
対策

頻出 ★★★

**Q2・WebAuthn**

パスキーのフィッシング耐  
性

頻出 ★★★

**Q3・OIDC**

**ID トークン(JWT)**の検証

頻出 ★★★

**Q4・OCSP**

**OCSP** ステージング

頻出 ★★★

**Q5・RFC 8705**

**mTLS** と証明書バインドト  
ークン

頻出 ★★★

 基礎概念 5枚  問題演習 5問  統合サマリー  2026 春・秋 想定

※ 過去問傾向(令和3～7年)と最新動向をもとにAIが作成した予想問題です。実際の試験問題ではありません。最終確認を推奨します。



## 学習マップ — 5問の相互関係

MAP

「フロント認証 → 認可コード → 認証連携 → 失効確認 → サーバ間信頼」の動線

Q1

### PKCE

公開クライアントの認可コード横取りを防ぎ、正規主体だけがトークン交換できる

RFC 7636 / OAuth 2.0



Q2

### パスキー

オリジンを署名対象にバインドしRPが検証。偽ドメインで署名検証不成立

FIDO2 / WebAuthn



Q3

### OIDC

IDトークン(JWT)で「誰が認証されたか」を伝達。  
iss/aud/exp/nonce検証

OIDC / 認証と認可



Q4

### OCSP

証明書の失効確認。ステープリングでプライバシー・可用性を改善

RFC 6960 / PKI



Q5

### mTLS

相互TLS + 証明書バインドトークンで盗難トークンの再利用を阻止

RFC 8705 / ゼロトラスト

### STEP 1 | WHY

「認証(誰か)・認可(何を許可)・PKI(鍵/証明書)」の3層で全体像を掴む

### STEP 2 | HOW

5問で「どの層・どのRFCの話か」を判別し、混同狙いの選択肢を切る

### STEP 3 | WHAT

空-雨-傘で対策(オリジン束縛・鍵所有証明・完全一致・失効確認)へ落とす

## ① 基礎概念 — 認証・認可・PKI 全体マップ

「通信の主体(層)」×「認証/認可/鍵基盤」の2軸で本デッキ5問を配置

CONCEPT

| 通信の主体 / 層         | 認証 (誰か)                                   | 認可・トークン (何を許可)                       | 証明書・鍵基盤 (PKI / 暗号)       |
|-------------------|-------------------------------------------|--------------------------------------|--------------------------|
| 利用者 ⇄ Webサイト (RP) | Q2 パスキー(FIDO2/WebAuthn)/ジェイルブレイク不可のオリジン束縛 | Q1 認可コードフロー + PKCE(RFC 7636)         | TLSサーバ証明書                |
| 認証連携 IdP ⇄ RP     | Q3 OIDC IDトークン(JWT)/認証層                   | OAuthアクセストークン(RFC 6749/6750, Bearer) | JWT署名 / JWKS(kid で公開鍵取得) |
| サーバ ⇄ サーバ / 基盤    | Q5 mTLS(相互TLS)でクライアント認証                   | 証明書バインドトークン(RFC 8705)/DPoP(RFC 9449) | Q4 失効確認 OCSP/CRL・PQC 移行  |

核心: 論点は「①どの層の通信か(フロント/認証連携/サーバ間)」と「②何の話か(認証=誰か/認可=何を許可/PKI=鍵・証明書)」で切り分けられる。認証(OIDC IDトークン)と認可(OAuthアクセストークン)の混同、および失効確認方式(OCSP/CRL/ステープリング)の取り違えが4択の定番ひっかけ。

## ② 基礎概念 — 混同しやすい概念 比較表

似て見えるが「役割・経路・対策」が異なる5ペア

CONCEPT

| 対象概念           | ● 概念A                                    | ● 概念B (混同パターン)                           |
|----------------|------------------------------------------|------------------------------------------|
| トークンの役割        | OIDC IDトークン = 認証「誰が認証されたか」をRPへ伝達         | OAuthアクセストークン = 認可「何にアクセス可か」。認証as認可の混同   |
| 失効確認方式         | OCSPステープリング = サーバが署名済み応答をTLSに添付し提示       | 従来OCSP = クライアントがレスポンスへ直接問合せ、アクセス先が漏れる    |
| トークン束縛         | 送信者制約トークン = 鍵所有の証明にバインド(cert-bound/DPoP) | ペアラトークン = 所持者なら誰でも利用可、盗難再利用に弱い           |
| パスキー種別         | device-bound passkey = 鍵が認証器内に留まり攻撃面が狭い  | synced passkey = クラウド同期でアカウント乗っ取り時に一括危険化 |
| redirect_uri登録 | 完全一致(exact)登録 = コード送信先を厳格に固定             | 前方一致(部分一致) = コード横取り/オープンリダイレクト悪用         |

**PKCE Proof Key for Code Exchange**

認可要求ごとに code\_verifier(乱数)とハッシュ code\_challenge(S256)を用い、認可コード横取り者の交換を阻止(検知でなく交換のbinding)。

 **RFC 7636 / OAuth 2.0**

**OIDC IDトークン OpenID Connect**

OAuth上の認証層。JWTで署名されたJWTで「誰が認証されたか」を伝達。iss/aud/exp、要求時はnonceを検証して初めて信頼可。

 **OIDC Core / JWT(RFC 7519)**

**証明書バインドトークン certificate-bound**

アクセストークンをmTLSクライアント証明書のサムプリント(cnfのx5t#S256)に束縛。秘密鍵なしでは盗難トークンを再利用できない。

 **RFC 8705**

**JWKS / kid JSON Web Key Set**

署名検証用の公開鍵集合。JWTヘッダのkidで対応鍵を選択。複数鍵の並行公開で無停止の鍵ローテーションが可能。

 **RFC 7517**

**FIDO2 / WebAuthn パスキー**

clientDataにオリジン、authenticatorDataにrpIdHashを含めて認証器が署名。RPがオリジン一致を検証し偽サイトでは不成立=フィッシング耐性。

 **W3C WebAuthn / CTAP**

**OCSPステープリング OCSP Stapling**

サーバがCA(レスポнда)から取得した署名済みOCSP応答をTLSハンドシェイクに添付。署名者は常にCA。プライバシー・可用性を改善。

 **RFC 6960 / RFC 6066**

**DPoP Demonstrating PoP**

クライアント保有鍵で署名したDPoP証明をリクエストに付与し、トークンを鍵所有に束縛する送信者制約方式。ベアラ盗難再利用を防ぐ。

 **RFC 9449**

**耐量子暗号(PQC) Post-Quantum**

ショアのアルゴリズムでRSA/ECCが危殆化。格子ベース(ML-KEM等)へ移行。計算量的仮定に依存し情報理論的安全ではない。

 **FIPS 203 / CRYPTREC**

「認証／認可／トークン束縛／失効／PQC」の根拠を取り違えない

| 規格 / RFC                                        | 種別         | 要点 (試験での問われ方)                                                      |
|-------------------------------------------------|------------|--------------------------------------------------------------------|
| <b>RFC 7636</b> PKCE                            | 認可コード保護    | code_verifier/challenge(S256)でコード横取りを阻止。 <b>Q1の正解</b> 。公開クライアント向け。 |
| RFC 6749 / 6750 OAuth 2.0                       | 認可フレームワーク  | 認可コード・アクセストークン(Bearer)。あくまで <b>認可</b> であり <b>認証</b> ではない。          |
| OpenID Connect Core                             | 認証層        | OAuth上のIDトークン(JWT)で <b>認証</b> 。iss/aud/exp/nonce検証。 <b>Q3の正解</b> 。 |
| W3C WebAuthn / FIDO2                            | パスワードレス認証  | オリジンを署名対象に束縛しRPが検証=フィッシング耐性。 <b>Q2の正解</b> 。                        |
| <b>RFC 8705</b> (mTLS) / <b>RFC 9449</b> (DPoP) | 送信者制約トークン  | cnf(x5t#S256)/鍵所有証明でトークンを束縛。 <b>Q5</b> ・午後の狙い目。                    |
| RFC 6960 OCSP / RFC 5280 CRL                    | 証明書失効      | OCSP=個別問合せ、CRL=一覧DL、ステープリング=TLS添付。 <b>Q4</b> 。                     |
| FIPS 203 ML-KEM / CRYPTREC                      | 耐量子暗号(PQC) | ショアでRSA/ECC危殆化→格子ベース移行。移行期はハイブリッド鍵交換が推奨。                           |

 **頻出の軸:** 「認証=OIDC/WebAuthn」「認可=OAuth」「トークン束縛=RFC 8705/9449」「失効=OCSP/CRL」「PQC=FIPS 203」。RFC番号と“何の話か”をセットで暗記すると4択のひっかけを外さない。





# 試験のコツ & 頻出ひっかけ

Exam Strategy & Common Pitfalls

TIPS

## ✓ 解くコツ

1. 認証か認可かで峻別: IDトークン(誰か)=OIDC/認証、アクセストークン(何を許可)=OAuth/認可。
2. フィッシング耐性の本質は「**オリジンを署名にバインドしRPが検証**」。OTPやパスワードハッシュは本質でない。
3. トークン盗難対策は「**鍵所有の証明に束縛(cert-bound/DPoP/mTLS)**」=送信者制約トークン。
4. PKCEは"検知"でなく"**交換の阻止(binding)**"。redirect\_uriは**完全一致**登録。

## ⚠ よくあるひっかけ

- パスキーを「**秘密鍵が漏れない**」だけで説明→それは鍵漏えい耐性で別論点、減点。
- OCSPステープリングの利点と従来OCSPの課題を取り違える。**署名者は常にCA**(Webサーバでない)。
- 「クッキー暗号化」「有効期限短縮」だけではトークン窃取再利用を緩和しきれない(束縛が主眼)。
- PQCは**情報理論的安全ではない**。AES-256はGroverでも実用上安全。ハイブリッド鍵交換は**推奨**(避けるは誤)。

合言葉: 「層(どこ)×役割(認証/認可/PKI)」で分類 → 対策は「**オリジン束縛・鍵所有証明・完全一致・失効確認・ハイブリッド移行**」の5語彙に集約される。

OAuth 2.0 の認可コードフローに PKCE(Proof Key for Code Exchange)を適用する主目的として、最も適切なものはどれか。

ア アクセストークンを JWT 形式にし、リソースサーバでの署名検証を可能にする

イ 認可サーバとリソースサーバ間の通信を相互 TLS で保護する

ウ 悪意のあるアプリによる認可コードの横取りを防ぎ、正規クライアント(元の認可要求を出した主体)だけがトークンと交換できるようにする

エ リフレッシュトークンの有効期限を延長し、再認可の頻度を下げる

正解 ウ ア=JWTアクセストークンの話/イ=mTLSの話/エ=PKCEの目的ではない

#### ▲ 空 (事実)

ネイティブ/SPAなど公開クライアントでは、認可コードを横取り(Authorization Code Interception)され得る。

#### ● 雨 (解釈)

要求ごとにcode\_verifier(高エントロピー乱数)を生成しcode\_challenge(S256)を送付、交換時にverifierを提示して束縛。本質は「検知」でなく交換の阻止。

#### ↑ 傘 (対策)

PKCE適用で、コードを横取りした第三者はcode\_verifierを知らずトークン交換できない。



FIDO2/WebAuthn によるパスキー認証がフィッシングに耐性をもつ根本的な理由として、最も適切なものはどれか。

ア 認証のたびにワンタイムパスワードを生成し、一定時間で失効させるため

イ 署名対象データにアクセス先のオリジン情報がバインドされ、RP がオリジンの一致を検証するため、偽ドメインでは署名検証が成立しない

ウ 秘密鍵をサーバ側にも複製保管し、突合により本人性を確認するため

エ 利用者が入力したパスワードをソルト付きハッシュで保存するため

正解 イ ア=OTPはAiTM中継で突破され得る/ウ=秘密鍵は認証器内に留まる(誤)/エ=本質でない

#### 空 (事実)

clientDataに乱数チャレンジとオリジン、authenticatorDataにrpIdHashが含まれ、認証器がそのハッシュに署名する。

#### 雨 (解釈)

RPは署名検証時にオリジン/rpIdHashの一致を検証。オリジンの異なる偽サイトでは検証に失敗し、資格情報もrpIdのオリジンでしか呼ばれない。

#### 傘 (対策)

パスワードレス+オリジン束縛のパスキーへ移行。秘密鍵はサーバへ渡らず認証器内に留まる。

実務メモ: AiTM(中間者型)フィッシングではOTPも中継突破される。「秘密鍵が漏れない」だけでは説明不足で、オリジンを署名にバインドしRPが検証と書けるかが分水嶺。

OpenID Connect(OIDC)における ID トークンに関する記述のうち、適切なものはどれか。

ア ID トークンは JWS で署名された JWT であり、iss・aud・exp・(要求時は)nonce 等のクレームを検証して初めて認証結果として信頼できる

イ ID トークンは認可(アクセス権の委譲)そのものを表し、リソースアクセスに直接使用する

ウ ID トークンの署名検証には、クライアントとサーバで事前共有した対称鍵のみを用いる

エ ID トークンは平文の JSON であり、TLS 上で送るため署名は不要である

正解 ア イ=アクセストークンの役割(認証/認可の混同)/ウ=非対称鍵(RS256等)が一般/エ=改ざん検知に署名必須

#### ▲ 空 (事実)

OIDCはOAuth 2.0上に構築した「認証」層で、IDトークン(JWT)により「誰が認証されたか」を伝える。

#### ● 雨 (解釈)

受信側RPは署名(通常RS256、JWKSからkidで公開鍵取得)とiss/aud/expを必ず検証し、リプレイ防止に要求時はnonceも照合する。

#### ✦ 傘 (対策)

検証項目の徹底を運用に組み込む。認証(IDトークン)と認可(アクセストークン)を混同しない。

電子証明書の失効確認方式である **OCSP** と、そのプライバシー・可用性上の課題を改善する **OCSP ステープリング**に関する記述のうち、適切なものはどれか。

ア OCSP ステープリングでは、クライアントが検証対象サーバごとに OCSP レスポンドへ直接問い合わせるため、利用者のアクセス先が漏れる

イ **OCSP ステープリング**では、サーバがあらかじめ取得した有効期限付きの署名済み **OCSP レスポンス**を **TLS ハンドシェイク**時にクライアントへ提示する

ウ OCSP は CRL 全体をダウンロードして失効を確認する方式であり、証明書ごとの個別問い合わせはできない

エ OCSP ステープリングを用いると、失効情報の署名者は当該 Web サーバ自身になる

**正解 イ** ア=従来OCSPの課題(ステープリングでない)/ウ=CRLの説明/エ=署名者はCA(OCSPレスポンド)

#### ▲ 空 (事実)

サーバがCA(OCSPレスポンド)から取得した署名済みレスポンスをTLSハンドシェイクに添付(staple)し、Certificate Statusとして提示する。

#### ● 雨 (解釈)

クライアントがレスポンドへ直接問い合わせないため、アクセス先の漏えいを防ぎ、レスポンドの負荷・可用性依存も下げる。署名者はあくまでCA。

#### ✦ 傘 (対策)

ステープリング採用でプライバシーと可用性を両立。従来OCSP直問合せ/CRL一覧DLとの違いを整理。

サーバ間 API 通信で mTLS(相互 TLS)と、証明書にバインドしたアクセストークン(certificate-bound token, RFC 8705)を組み合わせる主目的として、最も適切なものはどれか。

ア トークンが漏えいしても、対応するクライアント証明書の秘密鍵を持たない第三者は当該トークンを使えないようにする

イ トークンのペイロードを暗号化し、リソースサーバ以外が内容を読めないようにする

ウ トークンの有効期限を無期限にし、再発行を不要にする

エ クライアントがパスワードを送信することで多要素認証を実現する

正解 ア イ・ウ・エはいずれも目的ではない

#### ▲ 空 (事実)

アクセストークンを、mTLSで提示したクライアント証明書のサムプリント(確認方法クレーム cnf の x5t#S256)に束縛する。

#### ☂ 雨 (解釈)

リソースサーバは提示時のmTLS証明書のハッシュとcnfを照合。盗まれても正規証明書の秘密鍵がなければ利用できず、ベアラ盗難再利用(リプレイ)を防ぐ。

#### ☂ 傘 (対策)

ゼロトラストのサービス間通信で、送信者制約トークン(cert-bound / DPoP)を採用してトークン窃取の被害を封じめる。



## 午後 記述式 — シナリオ要点

パスワードレス認証導入／外部API連携の OAuth・PKI 運用

記述

### ◆ 問1 (春想定) 社内SaaSパスワードレス認証

- 設問1: AiTMでパスワード+SMS OTP突破・クッキー窃取 → パスキー移行。オリジンを署名対象にバインドしRPが検証するため偽ドメインで署名検証不成立。
- 設問2: セッション乗っ取り残存 → 送信者制約トークン(RFC 8705 cert-bound / RFC 9449 DPoP / DBSC)で鍵所有証明に束縛し窃取再利用を防止。
- 設問3: synced passkeyは同期クラウドアカウントが防御境界となり、乗っ取り時に複数デバイスが一括危殆化。

### ◆ 問2 (秋想定) 外部API連携 OAuth + PKI

- 設問1: JWT検証は署名 + **aud**(未検証=トークン転用/混同) + **exp**(未検証=失効/期限切れ受理継続)。iss/nbfでも可。
- 設問2: JWKSに**kid**付き複数鍵を並行公開、新旧のオーバーラップ期間+キャッシュTTL考慮で無停止ローテーション。
- 設問3: redirect\_uri前方一致 → 認可コード横取り/オープンリダイレクト悪用。是正策は**完全一致(exact)**登録。

採点の分水嶺: ①フィッシング耐性は「**オリジン署名バインド + RP検証**」(鍵漏えい耐性と区別) ②残存対策は「**鍵所有証明への束縛(送信者制約)**」 ③JWTは**aud/exp**まで検証 ④redirect\_uriは**完全一致**・鍵は**kid**で**無停止ローテーション**まで踏み込めているか。



# 統合サマリー — 頻出ヒートマップ & 総括

出題可能性 × 難易度で暗号・認証・PKI論点を俯瞰

SUMMARY

| 論点                       | 出題可能性 | 難易度 | 対策語彙              |
|--------------------------|-------|-----|-------------------|
| PKCE / 認可コード保護(RFC 7636) | 最有力   | 中   | verifier束縛・完全一致   |
| パスキー(FIDO2/WebAuthn)     | 最有力   | 中   | オリジン署名バインド        |
| OIDC IDトークン検証            | 最有力   | 中   | iss/aud/exp/nonce |
| OCSP / ステープリング           | 高     | 低   | 署名者はCA            |
| mTLS / 送信者制約トークン         | 高     | 高   | cert-bound/DPoP   |
| PQC移行(ショア/ML-KEM)        | 高     | 中   | ハイブリッド鍵交換         |

**総括:** 本分野は午前IIの定義問題(PKCE・OCSPステープリング・RFC 8705・ショア等)と、午後の実務シナリオ(パスワードレス認証・OAuth/PKI運用)の両面で狙われる。既存Web/暗号知識と接続する認証と認可の峻別、フィッシング耐性の本質=オリジン束縛、トークン盗難対策=鍵所有証明への束縛を最優先で固めること。

※本デッキはAIが過去問傾向・最新動向をもとに作成した予想問題です。正解・解説・図解は最終確認を推奨します。